



Stages

Stage 1 (5 pts.)

Stage 2 (8 pts.)

Stage 3 (6 pts.)

Stage 4 (5 pts.)

Starting code

L7: Father Laurence

Verona, a city full of romantic secrets and complicated intrigues. This is where the tragic story of Romeo and Juliet unfolded - lovers from feuding families. After the unfortunate events described in the movie "Romeo + Juliet" (1996), Father Laurence, the wise Franciscan and confessor to both lovers, began considering new methods of communication.

Inspired by 20th-century technological achievements, Father Laurence decided to use stream sockets (`SOCK_STREAM`) to transfer messages between lovers from the feuding families. He concluded that this modern form of communication might be more reliable and secure than traditional methods, such as messengers or monks sneaking around at night.

Students of the MiNI faculty are entrusted with the task of helping Father Laurence create a reliable communication system server based on stream sockets and `epoll`, which would allow connecting lovers sentenced to separation due to the hatred between their families and ensure a secure exchange of information.

Important: Complete messages (including those containing names) in the entire task always end with a newline character `\n`. No message from a client (including `\n`) can exceed `MAX_MSG_LEN`.

Below are the clients that can connect to the server. All, except `base_client.sh`, wait a certain number (default value 1) of **seconds** before sending each message; after sending the last message, netcat waits twice as long before closing the connection.

- **Basic client:** connects netcat to the UNIX socket `Laurenty` (constant `UNIX_SK_NAME` in the code) and prefixes each printed line with its name, does not send messages

independently.

```
./base_client.sh [name] [netcat args]
```

- **Client that waits and sends one incomplete message** (its name):

```
./intr_client.sh <name> <beloved> [seconds between messages]
```

- **Client that sends one complete (name) and one incomplete message** (beloved's name):

```
./intr_client2.sh <name> <beloved> [seconds between messages]
```

- **Client that correctly sends both names:**

```
./named_client.sh <name> <beloved> [seconds between messages]
```

- **Clients that correctly send both names and several more messages:**

```
./[chatty_client.sh | happy_client.sh] <name> <beloved> [seconds between messages]
```

Stages

Stage 1 (5 pts.)

Father Laurence wants to estimate how many young people need his help in passing secret love messages. Create a UNIX-type stream socket. Use the `epoll`, `poll`, or `select` mechanisms to listen for incoming connections from clients. After establishing a connection, print **"Another young person ([descriptor]) needs my help!"** and close the connection.

The program takes one argument: the number of seconds since the last connection was accepted, after which the server stops listening, prints **"No one needs my help anymore!"**, releases all resources, and terminates.

Before the program ends, remove the link to the UNIX socket from the file system. Leave no traces of your activity in Verona!

Example test: `./base_client.sh`

Stage 2 (8 pts.)

Accept declarations of love from people who wish to follow in the footsteps of the famous lovers. In the first message, the client prints their own name, and in the next, the name of their beloved. Once both fields are filled, print **"[client name] wants to marry [beloved name]"** and close the connection. You can assume that one name will not be used multiple

times.

Use the multiplexing mechanism added in the first stage to wait for messages. This time, Father Laurence waits `timeout` seconds for both incoming connections and messages from clients before terminating.

If a client disconnects before providing both names, print **"I lost contact with [client name]"**. If the client didn't manage to provide their own name, use a conspicuous placeholder string, e.g., **"??"**.

Example tests:

- `./named_client.sh Julia Romeo`
- `./intr_client.sh Romeo Julia`
- `./intr_client2.sh Walencjo Sylwia`

Stage 3 (6 pts.)

After obtaining both names from a client, try to find their beloved among the other clients by cross-comparing client names and beloved names. When successful, print **"[client name] and [beloved name] got married!"**, then send the message **"Congratulations, [client name] and [beloved name]!"** to both, close both connections, and release the associated resources. The sent string can be the same for both. If a pair cannot be found, continue the program's operation without changes.

Example test: `./named_client.sh Julia Romeo & ./named_client.sh Romeo Julia`

Stage 4 (5 pts.)

In Verona, young couples are forced to keep their love secret from their families. Father Laurence acts as an intermediary, passing messages between the lovers. When the server receives a complete message (terminated with a newline character) from a client, its task is to send it to the partner. If there is no partner, the message is discarded.

Example test: `./happy_client.sh Julia Romeo & ./chatty_client.sh Romeo Julia`

Sometimes Father Laurence finds messages without a signature, but with an addressee. He passes those as well. In the program, also listen (in `epoll`) on standard input for messages of the form `<addressee>:<message>` and forward them to the appropriate addressee. In case of an incorrect addressee or a line in the wrong format, ignore it and

print an appropriate error message.

Starting code

- [Makefile](#)
- [base_client.sh](#)
- [chatty_client.sh](#)
- [happy_client.sh](#)
- [intr_client.sh](#)
- [intr_client2.sh](#)
- [l7-common.h](#)
- [named_client.sh](#)
- [sop-werona.c](#)

Download as zip